ARM 32bit Architecture

* ARM does not manufacture physical chips, instead, it design the architecture and licenses it to companies like Apple, Qualcomm, & STMicroelectronics
* where It all Began In the 1980s, Acorn computers needed a new Processor for their business computers. finding nothing nothing suitable on the market, they decided to build their own based on a relatively new academic concept from Berkeley & stanford.

Intro

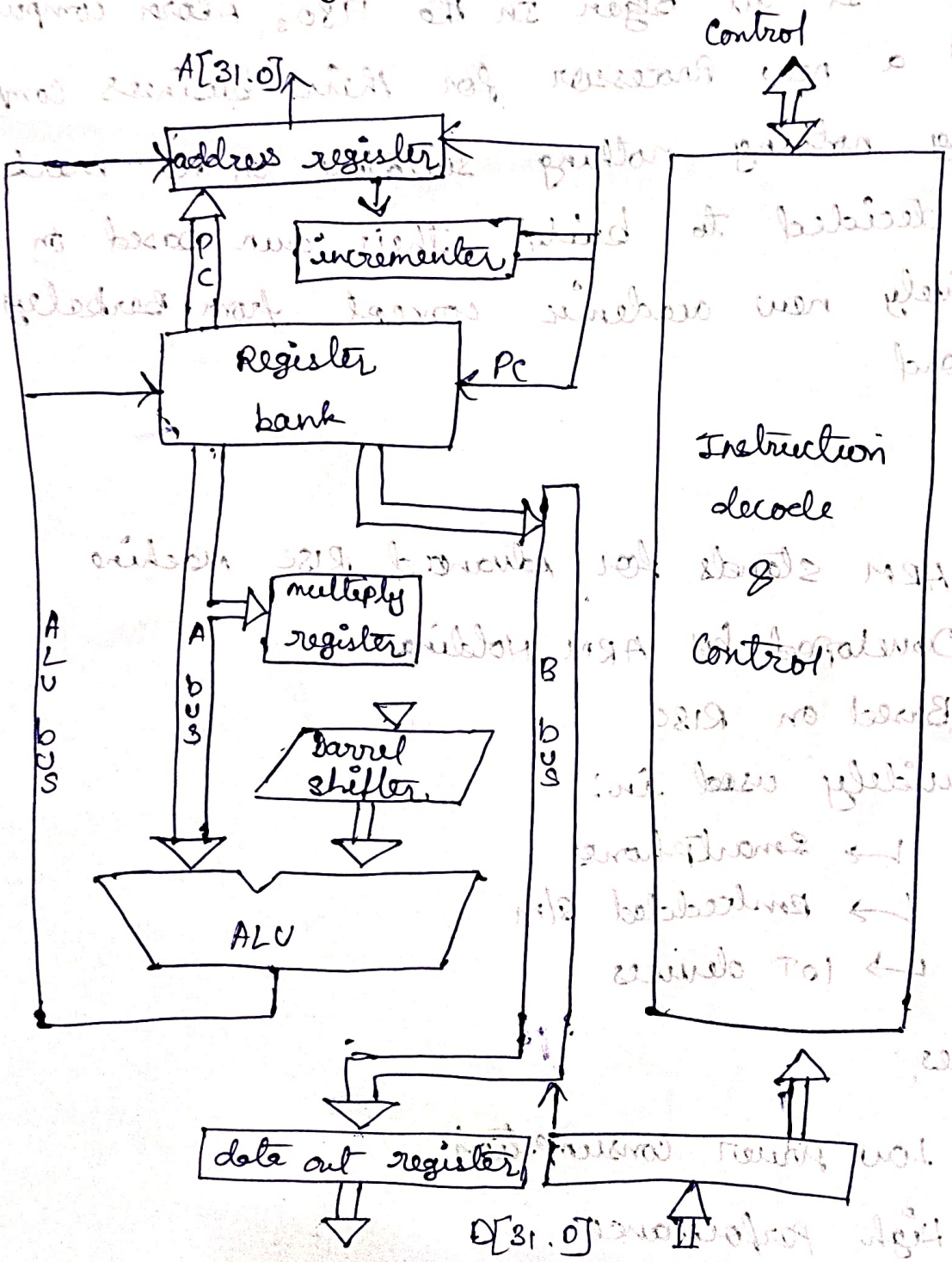
* ARM stands for Advanced RISC Machine
* Developed by ARM Holdings
* Based on RISC
* Widely used in:
  - ↳ Smartphones
  - ↳ Embedded S/m
  - ↳ IOT devices

Features

* Low Power Consumption
* High performance
* Small chip size
* Cost - effective
* Supports both 32-bit & 64-bit

ARM Architecture

* Load / Store architecture
* Large register set
* Fixed instruction length
* Pipelining support
* Thumb instructions set



Registers

* Total 16 register
  - R0-R12 → General Purpose
  - R13 → Stack Pointer (SP)
  - R14 → Link Register (LR)
  - R15 → Program Counter (PC)
  - CPER → Current Program Status Register

ARM Processor Modes

* user mode
* FIQ (Fast Interrupt)
* IRQ (Interrupt)
* supervisor mode
* Abort mode
* undefined mode

Pipelining

* Fetch
* Decode
* Execute
* memory
* write back

The Three Pillars - ARM Cortex Profiles

Modern ARM Processors are divided into three distinct families, easily remembered by the letters A, R, M:

* Cortex-A (APPlication): High-performance cores for complex OS (Linux, Android, iOS). Found in smartphones & servers.
* Cortex-R (Real-Time): Extremely low-latency cores for critical S/m where timing is absolute (automotive braking systems, hard drive controllers)

* Cortex-M (Microcontroller): Deeply embedded, highly deterministic & ultra-low power cores designed specifically for microcon.

~~Zooming In - The Cortex M Family~~

Cortex-M Lineup Lineup : The ultra-low power Tier

Cortex-M0 & M0+ → It is the smallest ARM Processors Replacing legacy 8-bit & 16-bit chips while sipping microwatts of power.

* Target : Battery-operated IoT Sensors, Smart wearables, & basic motor control.

* Key Trait : Extreme silicon efficiency. The M0+ has a highly optimized 2-stage pipeline

The Mainstream Tier

Cortex-M3 & M4 the workhorses of the embedded industry. These cores strike the perfect balance b/w Processing power, memory addressing, & power efficiency

* M3 Core : Bring 3-stage pipeline, hardware division & sophisticated interrupt handling

* M4 : It Takes the M3 & adds Digital signal Processing instructions & an optional Floating Point unit (FPU). Perfect for audio processing & complex math.

The Security Tier

Cortex-M23 & M33 as IOT devices became ubiquitous, security became paramount. These cores implement the ARMv8-M architecture.

* The Big Feature: they introduce TrustZone technology to the MCU level

* How it works: It physically partitions the processor's HW & memory into "Secure" & "Non-Secure" worlds, ensuring that even if the main application is hacked, encryption keys & secure boot processes remain untouched.

STMicroelectronics (STM32)

STM32 Nucleo F103RB

Intro :-

* Development board based on ARM Cortex-M4
* used for embedded sim & prototyping
* Has built-in Programmer (ST-LINK)

Specifications of the F446RE

* core Architecture : ARM Cortex-M4 with FPU
* Maximum clock speed : 180MHz (incredibly fast for a microcon)
* Flash memory (storage) : 512 K bytes
* SRAM (working memory) : 128 Kbytes

* operating vtg : 1.7 V to 3.6 V

Advantages

* High Performance
* Low Power
* Flexible Peripherals

Applications

* Robotics
* IOT
* Automation

Bare - Metal Vs HAL Programming

* H/w Abstraction Layer (HAL): Provided by STM cubeIDE. Easier to use, but adds overhead & hides underlying h/w operation

* Bare - Metal (Register - Level): writing directly to memory mapped register

* Advantages of Bare - Metal: Highly optimized code, lower memory footprint, complete understanding of Silicon behaviour & exact timing control

1) Bare - Metal Ex (Direct Register access):

```
// Blink LED connected to PA5
int main(void)

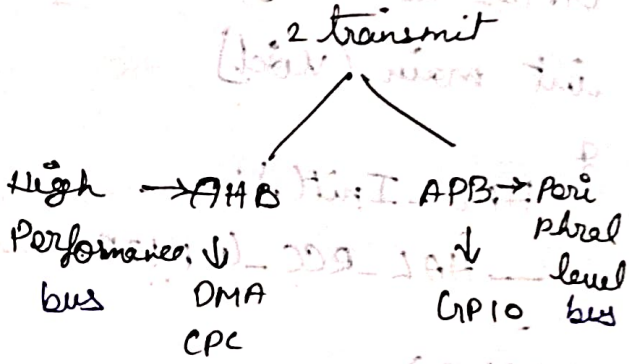
  RCC -> AHB1ENR |= (1<<0);

GPIOA -> MODER &= ~(3<<(5*2));
GPIOA -> MODER |= (1<<(5*2));
while(1)
{
  GPIOA -> ODR |= (1<<5);
for(volatile uint32_t i=0; i<500000; i++);
```

```
GPIOA -> ODR &= ~(1 << 5);
for (volatile uint32_t i = 0; i < 500000; i++);
}
}
```

what it does:-

* Enables GPIOA Clock
* Configures PA5 as O/P
* sets PA5 High → LED ON
* sets PA5 Low → LED OFF



2) CMSIS Ex (using CMSIS Definition)

```
int main (void)
RCC -> AHB1ENR |= RCC_AHB1ENR_GPIOAEN;

GPIOA -> MODER &= ~(GPIO_MODER_MODER5);
GPIOA -> MODER |= GPIO_MODER_MODER5_0;
while (1)
{
  GPIOA -> ODR |= GPIO_ODR_ODR5;
  for (volatile uint32_t i=0; i<500000; i++);
  GPIOA -> ODR &= ~GPIO_ODR_ODR5;
  for (volatile uint32_t i=0; i<500000; i++);
}
}
```

* uses CMSIS names for register & bits
* same functionality as bare-metal
* more readable & Portable

3) HAL ex (using HAL Library

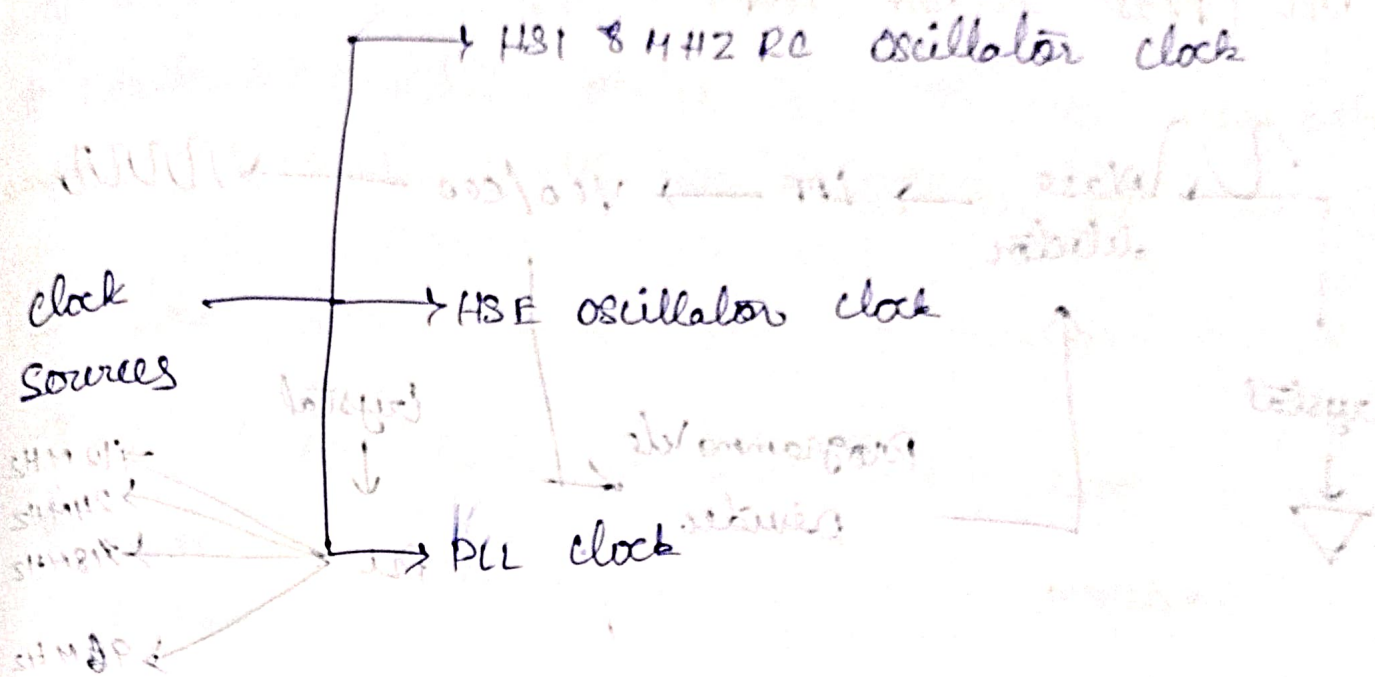
```
#included "stm32f4xx_hal.h".
GPIO_HandleTypeDef Led;
int main (void)
{
  HAL_Init();
  __HAL_RCC_GPIOA_CLK_ENABLE();

while(1)
{
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
  HAL_Delay(500);
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
  HAL_Delay(500);
}
}
```

* HAL_Init() initializes the s/m
* GPIO PA5 is configured as O/P using HAL
* HAL functions control the pin (H/L)
* Easy to use & portable across STM32 families

Clocks

- HSI — 16MHz low speed internal RC (LSI RC) → It drives the independent watchdog & optionally RTC used for auto-wake from stop/standby mode
- Additional Clock Sources
  - 32.768 KHz low speed external crystal (LSE crystal) → It optionally drives the real time clock. Each clock source can be switched on or off independently when it is not used to optimize power consumption



HSI oscillator clock (8MHz)

HSI → HSI stands for high speed Internal

Internal/External → HSI is an RC oscillator which is internal to the MCU.

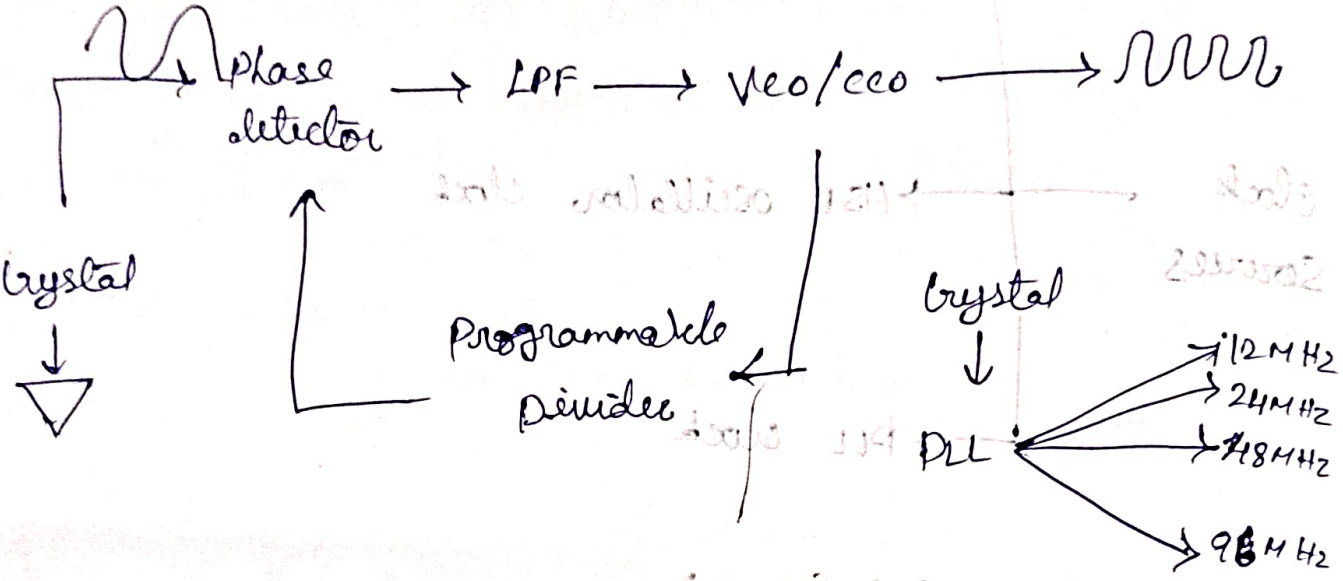
Source → The HSI clock sgnl is generated from an internal 8 MHz RC Oscillator

use → It can be used directly as a Sm. clock or divided by 2 to be used as PLL i/p

Purpose → MCU undergoes Power on reset or system reset by default, the MCU will get started with HSI clock as a main S/m. clock

Enabling → HSE or PLL can be enabled through code.

PLL (Phase Locked Loop)



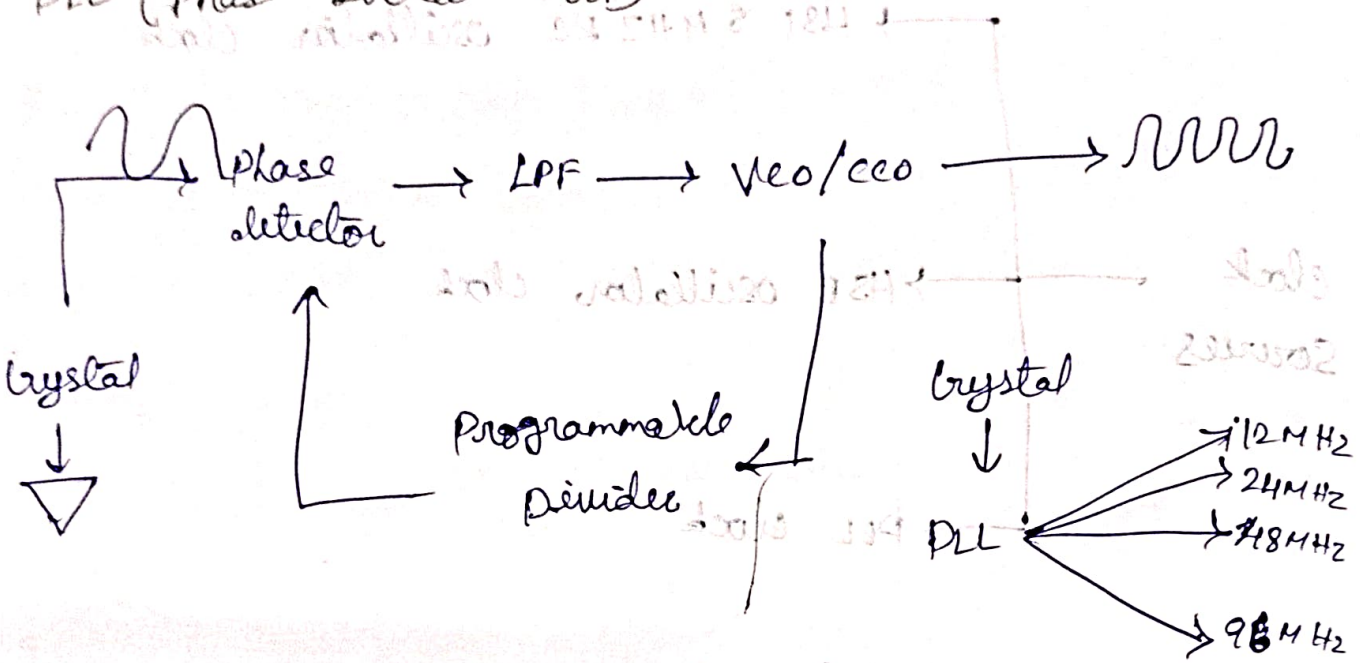
Key functions of a PLL

* Freq multiplication: Boosts a slow external crystal to high core speeds
* Clock synchronization: Aligns internal processor timing with external comm protocols like USB, CAN or Ethernet
* Jitter Reduction: Filters out high-freq noise to supply a clean, stable clock sgnl to the CPU & Peripherals

Core components

* Phase Detector (PD): Compares the reference i/p clock phase with the feedback clock phase to o/p an error sgnl.
* Low-Pass filter: Smooths the error sgnl into a steady Control Vtg
* Vtg-controlled Oscillator (VCO): Generates the o/p freq

PLL (Phase Locked Loop)



Key functions of a PLL

* Freq multiplication: Boosts a slow external crystal to high core speeds

* Clock synchronization: Aligns internal Processor timing with external comm Protocols like USB, CAN or Ethernet.

* Jitter Reduction: Filters out high-freq noise to supply a clean, stable clock sgnl to the CPU & Peripherals core components

* Phase Detector (PD): Compares the reference i/p clock phase with the feedback clock phase to o/p an error sgnl.

* Low-pass filter: Smooths the error sgnl into a steady Control Vtg

* Vtg-controlled Oscillator (VCO): Generates the o/p freq

proportional to the filtered control Vtg.

* Feedback Divider: Scales down the O/P freq before sending it back to the phase detector for locking

PLL Clock:-

* PLL → PLL stands for phase locked loop
* Internal/External → PLL is internal to the microcon
* Source → PLL source is HSI or HSE
* Use → It is the clock multiplier engine of the mcu. It can be used to boost or multiply the s/m clock freq higher than HSI or HSE
* Enabling → HSE can be enabled in pin configuration: reset & clock control (RCC) overview
* The RCC manages clocks for the core & all peripherals
* By default, peripherals are powered down to save power.
* Before using any peripheral (GPIO, Timer, ADC), its specific clock must be enabled in the RCC Register

S/m clock sources

* HSI (High-Speed Internal) - 16 MHz RC oscillator. Default clock on startup
* HSE (High-Speed External) - Crystal Oscillator
* PLL (Phase-Locked Loop) - Multiplies HSI or HSE to achieve max s/m speed

RCC Register Configuration

* RCC_CR - used to enable/disable HSI, HSE & PLL
* RCC_CFGR (Clock Config Register) - selects the S/m clock source & config bus prescalers.
* RCC_AHBIENR / RCC_APBIENR / RCC_APB2ENR - Peripheral clock enable Register used to turn on the clock for specific GPIO Ports, Timers, & the ADC.

Interrupt & the NVIC

Exceptions → S/m - level events generated by the ARM core (ex. HardFault, Sys Tick)

Interrupt → External events generated by peripheral

* Interrupts allow the Processor to execute main code & respond immediately to asynchronous events without Polling.

The Nested Vector Interrupt Controller (NVIC)

* Built directly into the ARM Cortex-M4 core.
* Handles routing of peripheral interrupts to the CPU
* Features: low latency, Prioritized interrupt handling and "nesting" (a higher Priority interrupt can Preempt a low Priority one)

Key NVIC Register

* NVIC_ISERx (Interrupt set-Enable Register): write a 1 to enable a specific interrupt line
* NVIC_ICERx (Interrupt clear-Enable Registers): write a 1 to disable a specific interrupt line

RCC Register Configuration

* RCC_CR - used to enable/disable HSI, HSE & PLL
* RCC_CFGR (Clock Config Register) - selects the S/m clock source & config bus prescalers.
* RCC_AHB1ENR / RCC_APB1ENR / RCC_APB2ENR - Peripheral clock enable Register used to turn on the clock for specific GPIO Ports, Timers, & the ADC.

Interrupt & the NVIC

Exceptions → S/m - level events generated by the ARM core (SVC, HardFault, Sys Tick)

Interrupt → External events generated by peripheral

* Interrupts allow the Processor to execute main code & respond immediately to asynchronous events without Polling.

The Nested Vector Interrupt Controller (NVIC)

* Built directly into the ARM Cortex-M4 core.
* Handles routing of peripheral interrupts to the CPU
* Features: low latency, Prioritized interrupt handling and "nesting" (a higher Priority interrupt can Preempt a low Priority one)

Key NVIC Register

* NVIC_ISER x (Interrupt set-Enable Register): write a 1 to enable a specific interrupt line
* NVIC_ICER x (Interrupt clear-Enable Registers): write a 1 to disable a specific interrupt line

* NVIC_ISPR x / NVIC_ICP
interrupt state.
* NVIC_IPR x (Interrupt
priority levels (0 to 15, wh
priority).

External Interrupt (EXIT)

* Allows GPIO Pins
* STM32F446RE has
others for RTC, USB, etc)
* Multiple Pins share
PB0, PC0 all connect to E
Time.

STM32F

Timer	TYPE	Bit width
TIM1	Advanced Control Timer	16-bit
TIM2	General purpose Timer	16 bit
TIM3	"	"
TIM4	"	"
SysTick	system Tick Timer	24-bit
IWDG	Independent watch Dog timer	16-bit
WWDG	window watch dog	16-bit

* NVIC_ISPRx / NVIC_ICPRx: Set or clear pending interrupt states.

* NVIC_IPRx (Interrupt Priority Register): Assign priority levels (0 to 15, where lower no mean higher priority)

External Interrupt (EXIT)

* Allows GPIO Pins to trigger interrupts
* STM32F446RE has 23 EXTI lines (16 for GPIO Pins others for RTC, USB, etc)
* Multiple Pins share the same EXTI line (e.g. PA0 PB0, PC0 all connect to EXTI0). only one can be active at a time.

STM32F103RB - Timer overview

Timer	Type	Bit width	No. of channels	Bus	Main use
TIM1	Advanced control Timer	16-bit	6	APB2	motor control, PWM Power electronics
TIM2	General purpose Timer	16-bit	4	APB1	General timing, IC/OC, PWM, counter
TIM3	"	"	"	"	"
TIM4	"	"	4	"	"
SysTick	system Tick Timer	24-bit	—	Core	RTOS tick, time base, delay.
IWDG	Independent watch DOG timer	16-bit	—	LSE (40)MHz Internal	s/m Protection reset on failure
WWDG	window watch dog	16-bit	—	APB1 (PCLK1)	windowed watchdog reset

STM32 Timer Calculation

Timer Clock (F_{TIM})
↓
Prescalar (PSC)
↓
Counter Clock ($F_{counter}$)
↓
Auto Reload Register (ARR)
↓
Timer Period/update Event

F_{TIM} (Timer Clock): the clock freq provided to the timer from the APB bus

PSC (Prescaler): A 16-bit register that divides the timer clock. The actual division factor is (PSC+1). It determines how fast the counter clock ticks

$F_{counter}$ (Counter Clock): the clock that increments the timer counter. It is the timer clk after Prescaler division

ARR: A 16-bit register that sets the max count value for the timer. The counter counts from 0 to ARR, then resets

Timer Period (T): Time taken to complete one full count from 0 to ARR & overflows

update Event: Event generated when the Counter overflows from ARR to 0

Main Formula

Time Period

$$T = \frac{(PSC+1)(ARR+1)}{F_{TIM}}$$

update Event Freq

$$F_{update} = \frac{F_{TIM}}{(PSC+1)(ARR+1)}$$

Find timer clock

SYS CLK → 72 MHz
AHB → 72 MHz
APB1 → 36 MHz
APB2 → 72 MHz

TIM2 ... TIM4 are on APB1. Because APB1 Prescaler is 2

$F_{TIM} = 2 \times PCLK1$

$= 2 \times 36\ MHz$

$F_{TIM} = 72\ MHz$

PSC Formula

$$F_{counter} = \frac{F_{TIM}}{PSC+1} \Rightarrow 1MHz = \frac{72MHz}{PSC+1}$$

$$\Rightarrow PSC+1 = \frac{72MHz}{1MHz} = 72$$

$$\boxed{PSC = 71}$$

ARR Calculation

Timer Period = 1ms

1 Count = 1 µs

1ms = 1000 µs = 1000 Counts

ARR + 1 = 1000 $\Rightarrow$ ARR = 999

Verify the calculation

$$T = \frac{(PSC+1)(ARR+1)}{F_{TIM}}$$

$$T = \frac{(71+1)(999+1)}{72,000,000}$$

$$T = \frac{72 \times 1000}{72,000,0000} = 0.001 =$$

$$\boxed{T = 1ms}$$

Quick Calculation

Desired Time	Tick (Counter clock)	PSC	ARR	Notes
1 µs	1MHz	71	0	1 Count = 1 µs
10 µs	"	"	9	10 Counts
100 µs	"	"	99	100 Counts
1 ms	"	"	999	1000 Counts
10 ms	"	"	9999	10,000 Count
100 ms	100 KHz	719	9999	10 µs/Count
1 sec	10 KHz	7199	"	100 µs / 4

Intro to STM32 Timers

* H/w blocks dedicated to counting clock cycles
* Basic Timers (TIM6, TIM7): simple - time-base generation, DAC triggering
* General purpose (TIM2 - TIM5, TIM9 - TIM14): I/P capture, O/P compare, PWM generation
* Advanced Timers (TIM1, TIM8): motor control, dead-time generation

Configuration EXTI & SYSCFG

* SYSCFG_EXTICRx: s/w configuration controller register route the specific GPIO port (A,B,C:..) to the EXTI line
* EXTI_IMR (Interrupt Mask Register): unmasks the specific EXTI line.
* EXTI_RTSR / EXTI_FTSR: Configuration the trigger edge (Rising Trigger selection Register / Falling TSR)
* EXTI_PR (Pending Register): must be cleared by s/w inside the Interrupt Service Routine (ISR) to prevent an infinite interrupt loop.

Time Architecture & Time Base unit

* counter (CNT): the Register that actually counts up or down
* Prescaler (PSC): Divides the incoming ~~clk~~ timer clk to slow down the counting rate
* Auto-Reload Register (ARR): Defines the max value the counter reaches before resetting

Calculating Timer freq.

* Timer Freq = Peripheral clk / (PSC + 1)
* Update Event (overflow) Freq = Timer Freq / (Auto-Reload + 1)

Key Timer Registers

* TIMx_CR1 (Control Register 1) - Enables the timer counter (CEN bit)
* TIMx_PSC : 16-bit Register to divide the clk
* " _ ARR : 16 bit or 32-bit register holding the count limit
* TIMx_SR (Status Register) : Contains the updates Interrup flag (UIF), which goes high when the timer overflow

Timer Interrupt

* TIMx_DIER (DMA / Interrupt Enable Register) : The UIE (Update Interrupt enable) bit must be set to allow the timer to generate a HW Interrupt upon overflow.
* Requires enabling the specific timer IRQ in the NVIC (NVIC_ISERx)
* Inside the ISR, the UIF bit in TIMx_SR must be cleared manually.

O/P Compare & PWM

* Timers can control physical O/P Pins automatically
* TIMx_CCMR1 / CCMR2 : Define PWM mode 1 or 2
* TIMx_CCRx : Determines the duty cycle by Comparing this value against the main CNT register

PWM Register Configuration Flow:-

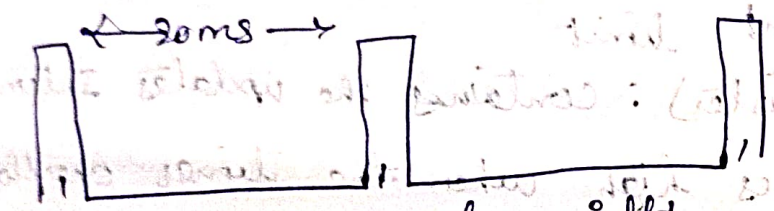
* Enable clk for timer & target GPIO
* Set GPIO Alternate Functions mode (MODER & AFR Register
* Configure Timer PSC & ARR for desired PWM freq
* Set PWM mode in CCMRx & duty cycle in CCRx
* Enable capture / compare O/P in TIMx_CCER & Start the timer.

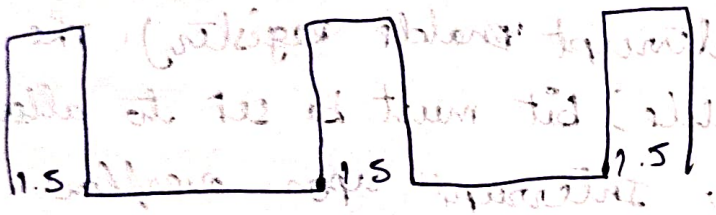
Servo motor

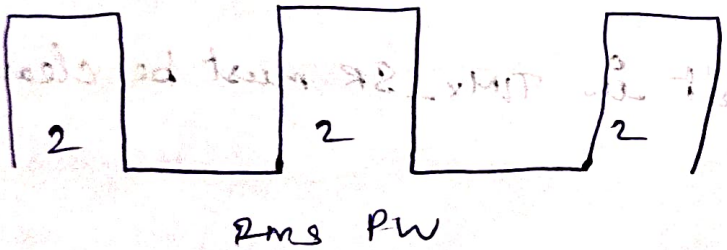
Red wire → +5V

Brown → GND

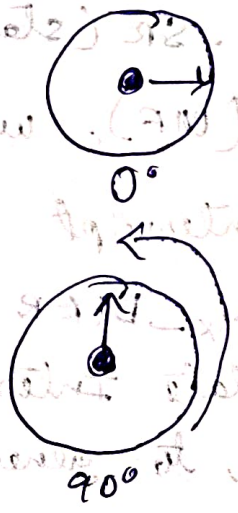
Orange → PWM







shaft rotates in degree



ADC overview, in STM32F446RE:

Translates continuous analog voltage into discrete digital values.

Features three 12 bit ADCs (ADC1, ADC2, ADC3)

Resolution is up to 12 bits (values from 0 to 4095).

Maximum operating voltage is V_DDA (typically 3.3V).

ADC conversion Modes:

single Conversion mode: ADC takes one reading and stops.

Continuous Conversion Mode: ADC takes continuous reading in the background.

Scan Mode: Converts a sequence of multiple channels automatically.

Can be triggered by software or hardware (TIM, EXTI).

Key ADC Control Registers:

ADC_CR1 (Control Register 1): Set resolution (12, 10, 8, or 6 bit) and enables Scan Mode.

ADC_CR2 (Control Register 2).

ADON bit: Turns on the ADC Module.

CONT bit: Enables continuous Conversion

SWSTART bit: Triggers a software Conversion

Channel selection and sampling Time:

ADC_SQRx (Regulare Sequence Register) Defines the order of channel to be converted and the total number of Conversions. For a single Channel, set it to SQR3.

ADC_SMPRx (Sample Time Regis) Configures how long
ADC samples the voltage before converting
(e.g., 3 cycles, 84 cycles). Longer sampling time
equal higher accuracy for high-impedance sour

ADC - without DMA vs with DMA:

(Flow without DMA)

ADC
↓
CPU reads
ADC values
↓
CPU stores value
in RAM.

with DMA.

ADC
↓
DMA Controller
↓
RAM Buffer
↓
Interrupt to CPU
Transfer Complet

Data path

ADC peripheral → CPU (Cortex-M) → RAM Buffer

CP stats ADC
Cpu waits for Conversion
Cpu reads the value
Cpu stores it in RAM
Repeat for every
Sample.
CPU is involved in every
transfer.

ADC → DMA → RA
Interrupt ←
to CPU
CPU define Configuration
ADC & DMA
DMA reads data from
ADC
DMA writes data to
RAM.
After 100 Samples,
DMA generates
interrupt.
Cpu is notified
(callback).
CPU is free during
transfer